

ПРАКТИЧНЕ ЗАНЯТТЯ №10

Тема: ДИНАМІЧНІ СТРУКТУРИ ДАНИХ

Звіт про виконання практичного заняття

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Оволодіння практичними навичками роботи з динамічними структурами даних, зокрема з однозв'язними та двозв'язними списками, а також закріплення навичок створення, перегляду, додавання і видалення елементів списку.

Теми для попереднього опрацювання

- Функції в мовах C/C++.
- Показчики.
- Файли в стилі C.
- Рядки.

Теоретичні відомості

Динамічні структури даних використовуються тоді, коли кількість елементів може змінюватися під час виконання програми. На відміну від статичних масивів, динамічні структури дають змогу виділяти та звільняти пам'ять у процесі роботи програми.

Список - це послідовність елементів, кожен з яких містить інформаційну частину та додаткову частину з показчиками. Інформаційна частина зберігає дані, а додаткова частина забезпечує зв'язок з іншими елементами списку.

В однозв'язному списку кожен елемент містить показчик лише на наступний вузол. Такий список можна переглядати тільки в одному напрямку - від початку до кінця. Для роботи з ним зазвичай використовують показчик на початок списку, а також за потреби показчик на останній елемент.

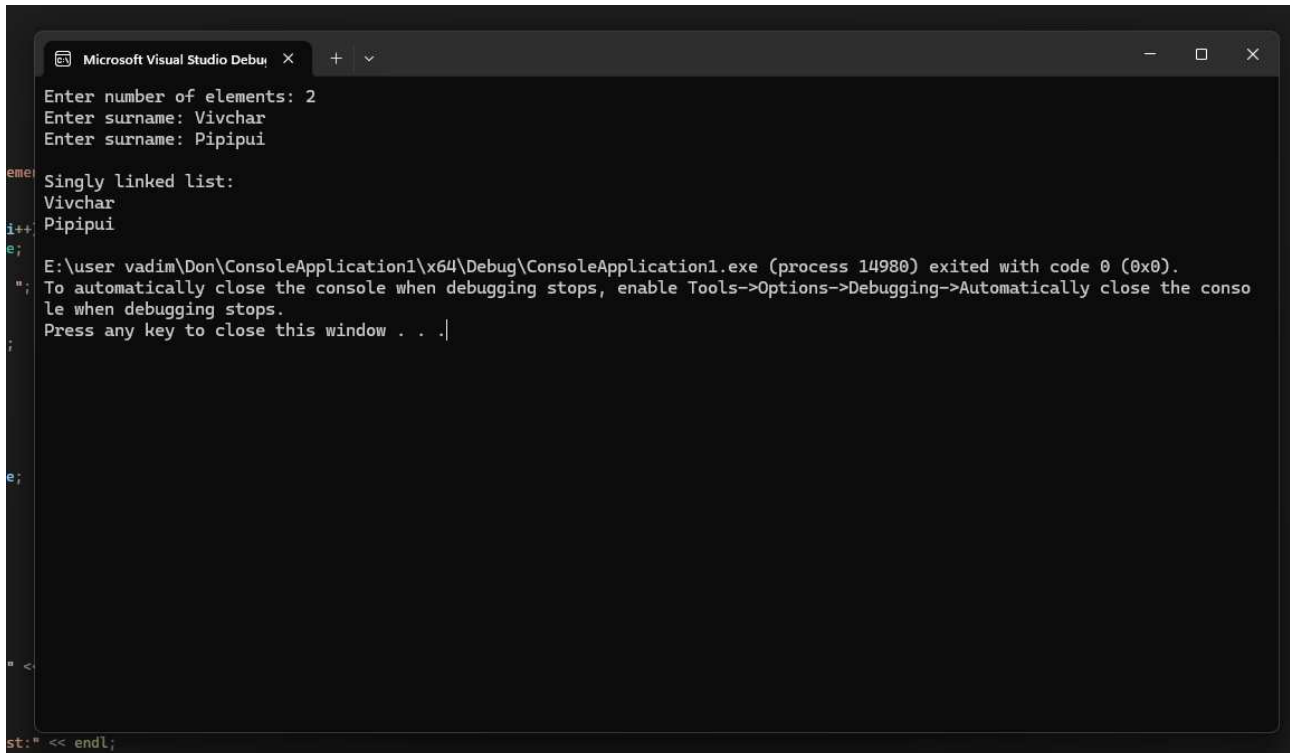
Двзв'язний список відрізняється тим, що кожен його елемент має два показчики: на наступний і на попередній вузол. Це дає змогу переглядати список як з початку, так і з кінця. Двзв'язний список зручніший для операцій видалення та вставки елементів, але потребує більше пам'яті через наявність додаткового показчика.

Хід роботи

Приклад 1. Створення та перегляд однозв'язного списку

У першому прикладі реалізовано створення однозв'язного списку. Користувач вводить кількість елементів і значення кожного елемента. Для

кожного нового вузла динамічно виділяється пам'ять, після чого вузол приєднується до кінця списку. Після створення список переглядається з початку до кінця.



```
Microsoft Visual Studio Debug
Enter number of elements: 2
Enter surname: Vivchar
Enter surname: Pipipui

Singly linked List:
Vivchar
Pipipui

E:\user vadim\Don\ConsoleApplication1\Debug\ConsoleApplication1.exe (process 14980) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 1 - Результат створення та перегляду однозв'язного списку

Приклад 2. Меню для роботи з однозв'язним списком

У другому прикладі створено меню для виконання основних операцій з однозв'язним списком. Програма дозволяє створити список, переглянути його, додати елемент на початок або в кінець, додати елемент після заданого ключа, а також видалити елемент з початку, з кінця або за ключем.



```
Enter number: 38
Menu:
1 - Create list
2 - Show list
3 - Remove from beginning
4 - Remove by key
5 - Remove from end
6 - Add to beginning
7 - Add to end
8 - Add after key
9 - Exit
Choose option: 2
List: 19 20 38

Menu:
1 - Create list
2 - Show list
3 - Remove from beginning
4 - Remove by key
5 - Remove from end
6 - Add to beginning
7 - Add to end
8 - Add after key
9 - Exit
Choose option: 7
Enter value: 40

Menu:
1 - Create list
2 - Show list
3 - Remove from beginning
4 - Remove by key
5 - Remove from end
6 - Add to beginning
7 - Add to end
8 - Add after key
9 - Exit
Choose option: 2
List: 19 20 38 40

Menu:
1 - Create list
2 - Show list
3 - Remove from beginning
4 - Remove by key
5 - Remove from end
6 - Add to beginning
7 - Add to end
8 - Add after key
9 - Exit
Choose option: 8
Enter key to remove: 20

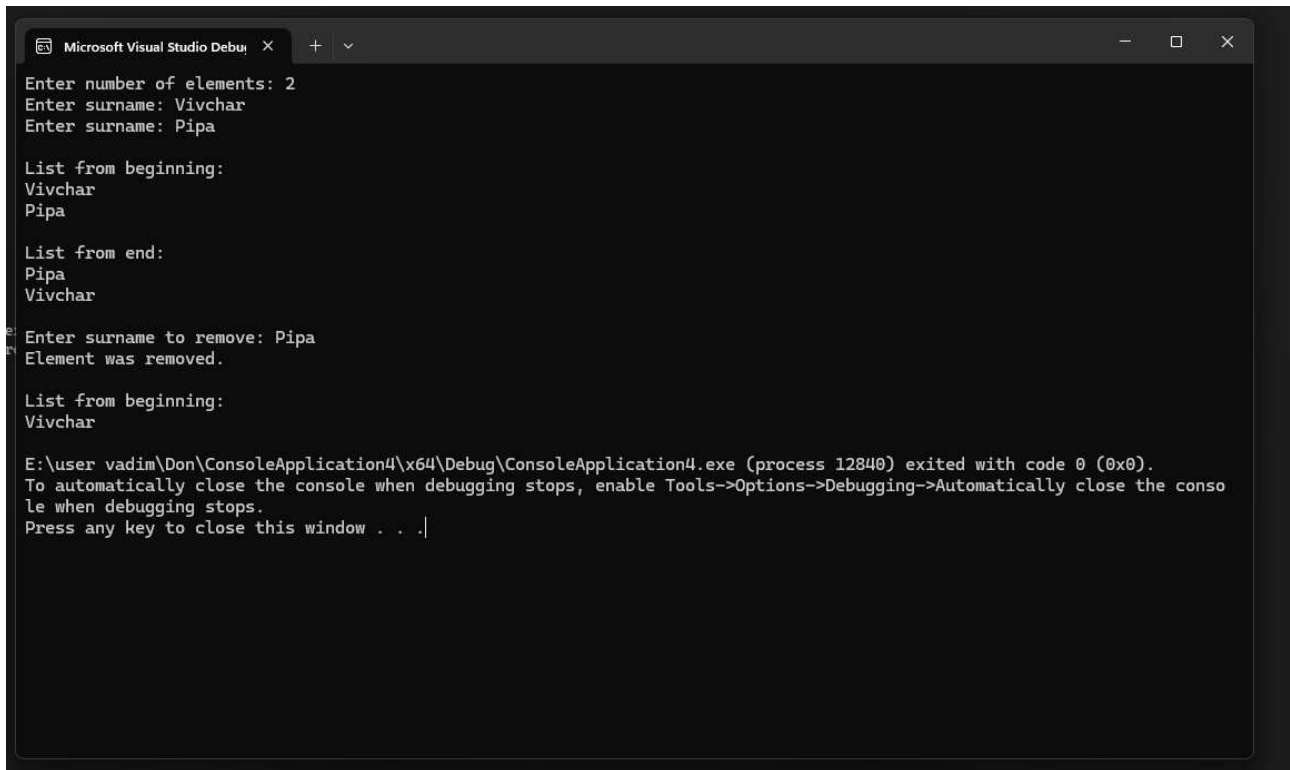
Menu:
1 - Create list
2 - Show list
3 - Remove from beginning
4 - Remove by key
5 - Remove from end
6 - Add to beginning
7 - Add to end
8 - Add after key
9 - Exit
Choose option: 9
Program finished.

E:\user vadim\Don\ConsoleApplication2\Debug\ConsoleApplication2.exe (process 7252) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 2 - Додавання, перегляд і видалення елементів однозв'язного списку

Приклад 3. Робота з двозв'язним списком

У третьому прикладі реалізовано двозв'язний список. Після введення елементів програма показує список у прямому напрямку, потім у зворотному напрямку. Далі виконується видалення елемента за заданим значенням і повторний перегляд списку після видалення.



```
Microsoft Visual Studio Debug Console
Enter number of elements: 2
Enter surname: Vivchar
Enter surname: Pipa

List from beginning:
Vivchar
Pipa

List from end:
Pipa
Vivchar

Enter surname to remove: Pipa
Element was removed.

List from beginning:
Vivchar

E:\user vadin\Don\ConsoleApplication4\x64\Debug\ConsoleApplication4.exe (process 12840) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 3 - Результат роботи з двозв'язним списком

Контрольні запитання та відповіді

1. Що таке список? З яких двох смислових частин складається структура, що використовується для організації списку?

Список - це динамічна структура даних, що складається з послідовності взаємопов'язаних елементів. Кожен елемент списку містить дві смислові частини: інформаційну частину, у якій зберігаються дані, та додаткову частину, у якій зберігаються покажчики на інші елементи списку.

2. Як оголосити елемент списку?

Елемент списку оголошується як структура, що містить поле для даних і покажчик на інший елемент. Наприклад, для однозв'язного списку структура може містити поле `data` та покажчик `next` на наступний елемент. Для двозв'язного списку додатково використовується покажчик `prev` на попередній елемент.

3. Який алгоритм створення однозв'язного списку?

Спочатку створюється покажчик на початок списку. Для кожного нового елемента виділяється пам'ять, заповнюється інформаційне поле, а покажчик `next` встановлюється у значення `nullptr`. Якщо список порожній, новий елемент стає першим. Якщо список уже містить елементи, останній елемент зв'язується з новим, а покажчик кінця списку переноситься на новий вузол.

4. Який алгоритм друку однозв'язного списку?

Для друку однозв'язного списку створюється допоміжний покажчик, який встановлюється на початок списку. Далі у циклі, поки покажчик не дорівнює `nullptr`, виводиться інформаційна частина поточного елемента, після чого покажчик переходить до наступного вузла.

5. Який алгоритм додавання елементу однозв'язного списку на початок, в кінець, в середину списку?

Для додавання на початок створюється новий вузол, його покажчик `next` встановлюється на поточний початок списку, після чого `head` переноситься на новий вузол. Для додавання в кінець потрібно пройти до останнього елемента і записати адресу нового вузла в його поле `next`. Для додавання в середину потрібно знайти потрібну позицію, зв'язати новий вузол з наступним елементом і змінити покажчик попереднього вузла на новий елемент.

6. Який алгоритм видалення елементу однозв'язного списку з початку, з кінця, з середини списку?

Для видалення з початку покажчик `head` переноситься на наступний елемент, а старий перший вузол видаляється з пам'яті. Для видалення з кінця потрібно знайти передостанній елемент, видалити останній вузол і встановити покажчик `next` передостаннього елемента в `nullptr`. Для видалення з середини потрібно знайти елемент перед тим, що видаляється, змінити його покажчик `next` так, щоб він пропускав вузол, який видаляється, а потім звільнити пам'ять цього вузла.

7. Чим відрізняються алгоритми створення та друку двозв'язного списку від однозв'язного?

Під час створення двозв'язного списку для кожного елемента потрібно встановити не лише покажчик `next` на наступний елемент, а й покажчик `prev`

на попередній. Друк двозв'язного списку можна виконувати у двох напрямках: від початку до кінця за допомогою next або від кінця до початку за допомогою prev.

8. Чим відрізняються алгоритми додавання та видалення двозв'язного списку від однозв'язного?

У двозв'язному списку під час додавання або видалення потрібно оновлювати два зв'язки: next і prev. Це робить алгоритм дещо складнішим, але водночас зручнішим, оскільки елемент має зв'язок не тільки з наступним, а й з попереднім вузлом. Завдяки цьому легше виконувати переміщення списком у двох напрямках і видаляти елементи з різних позицій.

Висновок

У ході виконання практичного заняття було розглянуто динамічні структури даних і принципи роботи зі списками. Було реалізовано створення та перегляд однозв'язного списку, меню для виконання основних операцій з однозв'язним списком, а також роботу з двозв'язним списком.

Практичні приклади показали, що списки є зручним засобом організації даних, коли кількість елементів може змінюватися під час виконання програми. Однозв'язний список забезпечує просту послідовну організацію елементів, а двозв'язний список дає можливість переміщуватися у двох напрямках і гнучкіше виконувати операції додавання та видалення.